

Question 1

Langages compilé et interprété :

Expliquez les différences entre un langage compilé (ex. : C) et un langage interprété (ex. : Python) en utilisant des exemples concrets. Comparez leurs méthodes d'exécution, performance et facilité de débogage.

EN C :

- Méthode d'exécution : Le code source est compilé avant exécution en un fichier binaire exécutable (machine code). Exemple : `gcc main.c -o programme` : cela crée `programme.exe` (binaire).
- Performance : Très rapide, car le code machine est directement exécuté par le processeur.
- Débogage : Plus complexe : le code est transformé avant exécution. Il faut souvent utiliser un debugger (gdb).
- Portabilité : Le binaire dépend du système d'exploitation et du processeur.
- Outils associés : Compilateur (gcc), Makefile.

EN PYTHON :

- Méthode d'exécution : Le code source est lu et exécuté ligne par ligne par un interpréteur. Exemple : `python3 script.py` : le code est lu et exécuté immédiatement.
- Performance : Plus lent, car chaque ligne est traduite et exécutée en temps réel.
- Débogage : Plus simple : les erreurs sont détectées à l'exécution, les messages d'erreur sont explicites.
- Portabilité : Le même script Python fonctionne sur tous les systèmes (si l'interpréteur est présent).
- Outils associés : Interpréteur (python3).

Rôle du compilateur et chaîne de compilation :

• Quel est le rôle d'un compilateur ?

Le compilateur sert à traduire le code source, écrit en C par exemple, en code machine. Il vérifie aussi la syntaxe, les types, et effectue des optimisations avant de produire un fichier exécutable.

• Décrivez brièvement les étapes principales de la compilation avec gcc et les options pour les visualiser. **Importance d'un Makefile :**

Les principales étapes internes de la compilation sont :

1. Préprocesseur
 - Gère les directives comme `#include` et `#define`.
 - Produit un code source "prétraité".
2. Analyse lexicale

- Découpe le code en unités lexicales (tokens) : mots-clés, identifiants, symboles, etc.
- 3. Analyse syntaxique
 - Vérifie la structure du code selon la grammaire du langage (ex. : parenthèses, blocs, etc.).
 - Produit un arbre syntaxique.
- 4. Analyse sémantique
 - Vérifie la cohérence logique : types corrects, variables déclarées, portée, etc.
- 5. Optimisation du code
 - Améliore le code intermédiaire pour le rendre plus rapide ou moins gourmand en mémoire.
- 6. Génération de code natif
 - Traduit le code intermédiaire en langage machine (fichier objet .o), puis effectue l'édition de liens pour produire l'exécutable.

Pour visualiser certaines étapes avec gcc :

Préprocesseur : `gcc -E fichier.c -o fichier.i`

Code assembleur : `gcc -S fichier.c -o fichier.s`

Code objet : `gcc -c fichier.c -o fichier.o`

Exécutable : `gcc fichier.o -o programme`

Affiche toutes les étapes : `gcc -v fichier.c -o programme`

• Comment un Makefile utilise-t-il les étapes de compilation ?

Un Makefile automatise ces étapes : il indique à make dans quel ordre compiler les fichiers et quelles dépendances (fichiers .h) doivent être prises en compte. Il exécute automatiquement les commandes gcc nécessaires selon les fichiers modifiés.

Utilité

- Évite de tout recompiler inutilement.
- Simplifie la gestion de projets avec plusieurs fichiers.
- Rend la compilation plus rapide, claire et organisée.

• Pourquoi est-il essentiel pour gérer efficacement un projet avec de nombreux fichiers .c et .h ?

Un Makefile est essentiel car il permet de gérer automatiquement la compilation d'un projet composé de nombreux fichiers source et d'en-tête. Sans lui, il faudrait recompiler manuellement chaque fichier à chaque modification, ce qui serait long et source d'erreurs.

Grâce à ses règles de dépendances :

- Il ne recompile que les fichiers modifiés, ce qui gagne du temps.
- Il organise clairement la structure du projet (chaque module a sa règle).

- Il facilite la maintenance et l'évolution du code.
- Il garantit une compilation cohérente, avec les mêmes options pour tous les fichiers.

QUESTION 2

Écrivez un programme en C qui prend en entrée le nom ou le chemin d'un fichier (fournis comme argument en ligne de commande). Ce programme doit fonctionner efficacement avec des fichiers de différentes tailles, de 1 Ko à 1 Go. Le programme doit calculer et afficher :

- Le nombre total de caractères dans le fichier.
- Le nombre total de lignes dans le fichier.
- Le nombre total de mots dans le fichier (mots séparés par des espaces, ponctuation, etc.).
- Le nombre total de valeurs numériques dans le fichier (constitués d'un ou plusieurs chiffres). Ex: 1, 34, 456

Indications supplémentaires :

- Optimisez la lecture pour manipuler de grandes tailles de fichiers.
- Gérer les erreurs telles qu'un fichier inexistant ou des autorisations insuffisantes.
- Testez avec des fichiers contenant une combinaison de texte, de chiffres, et de caractères spéciaux.

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <ctype.h>

int main(int argc, char* argv[]) { // Fonction qui compte le nombre de caractères, mots, lignes et valeurs numériques dans un fichier
    int fd, size;
    char content;

    if (argc != 2) {
        printf("Le nombre d'arguments rentrés est incorrect. \n");
        return 0;
    }

    char* nom_du_fichier = argv[1];

    int nb_char = 0;
    int nb_lign = 1;
    int nb_word = 0;
    int nb_int = 0;
    char old_content = '';

    fd = open(nom_du_fichier, O_RDONLY); // On ouvre le fichier en lecture seule
    if (fd == -1) {
        printf("Erreur : impossible d'ouvrir le fichier '%s'\n", nom_du_fichier);
        return 1; // Ou return -1 pour indiquer une erreur
    }

    while (1) {
        size = read(fd, &content, 1); // On lit le fichier octet par octet
        if (size < 1) {
            if (old_content != '' && old_content != '\n' && !ispunct(old_content)) {
                nb_word++;
            }
            break;
        }

        if (content != '\n') {
            nb_char++;
            if (!isdigit(content) && !isdigit(old_content)) {
                nb_int++;
            }
        }
        else {
            nb_lign++;
        }
        if ((content == ' ' || content == '\n' || !ispunct(content)) && (old_content != '' && old_content != '\n' && !ispunct(old_content))) {
            nb_word++;
        }
        old_content = content;
    }

    printf("Nombre total de caractères : %d \n", nb_char);
    printf("Nombre total de lignes : %d \n", nb_lign);
    printf("Nombre total de mots : %d \n", nb_word);
    printf("Nombre total de valeurs numériques : %d \n", nb_int);

    close(fd);
    return 0;
}
```

QUESTION 3

Modification de l'ordre séquentiel :

- Quelles sont les différentes manières de modifier l'ordre séquentiel d'exécution dans une fonction en C ?

Plusieurs structures de contrôle permettent de modifier l'ordre séquentiel :

- Structures conditionnelles: if, else if, else, switch. Permettent de **choisir** un chemin d'exécution selon une condition.
- Boucles: for, while, do ... while. Permettent de **répéter** une partie du code plusieurs fois.
- Saut: break, continue, return, goto. Permettent de **quitter** une boucle ou une fonction, ou de **sauter** à une autre instruction.
- Appels de fonctions. Permettent de **déléguer** une partie du travail à une autre fonction (et donc changer l'ordre logique d'exécution).

- Pourquoi ces techniques sont-elles importantes ? Expliquez avec des exemples.

Elles permettent de :

- **Contrôler la logique du programme** (par exemple : exécuter un code seulement si une condition est vraie).
- **Répéter efficacement des actions** sans réécrire le code.
- **Structurer** le programme en petites fonctions pour plus de clarté et de réutilisabilité.

Récursion :

- Pourquoi utilise-t-on la récursion (une fonction qui s'appelle elle-même) ?

La récursion est utilisée lorsqu'un problème peut être décomposé en sous-problèmes similaires au problème initial. Une fonction s'appelle elle-même jusqu'à atteindre un cas de base (condition d'arrêt).

Elle rend le code :

- Plus lisible et logique pour certains problèmes (factorielle, Fibonacci, parcours d'arbre, etc.)
- Mais parfois moins efficace (plus lente, plus de mémoire utilisée pour la pile d'appels).

Exemple pour la factorielle :

```
int factorielle(int n) {  
    if (n <= 1) return 1; // cas de base  
    else return n * factorielle(n - 1); // appel récursif  
}
```

- Comment convertir une fonction récursive en une version itérative ? Donnez un exemple illustrant cette conversion.

On peut transformer une fonction récursive en boucle (itérative) pour éviter les appels multiples et économiser de la mémoire.

Exemple pour la factorielle :

```
int factorielle_iterative(int n) {  
    int resultat = 1;  
    for (int i = 1; i <= n; i++)  
        resultat *= i;  
    return resultat;  
}
```

QUESTION 4

Différentes approches :

Expliquez comment créer un tableau 2D (tableau de deux dimensions) en C :

- En déclarant directement un tableau avec une taille fixe.

On déclare directement le tableau avec ses dimensions connues à la compilation.

✓ Avantages :

- Simple et rapide à déclarer.
- Mémoire gérée automatiquement.

⚠ Inconvénients :

- Taille **fixe** connue à l'avance.
- Peu flexible.

- En utilisant des pointeurs pour allouer de la mémoire dynamiquement.

On alloue de la mémoire à l'exécution avec malloc(), ce qui permet d'avoir un tableau dont la taille n'est connue qu'à l'exécution.

✓ Avantages :

- Taille du tableau décidée à l'**exécution**.
- Très utile pour les **tableaux de taille variable**.

⚠ Inconvénients :

- Nécessite de **libérer la mémoire** avec free().
- Légèrement plus lent et plus complexe à manipuler.

Programme :

Écrivez un programme en C pour démontrer ces deux approches, en accédant aux éléments et en affichant leurs valeurs.

- Méthode statique :

```
#include <stdio.h>
```

```
int main() {
    int tableau[2][3] = { {1, 2, 3}, {4, 5, 6} }; // tableau 2x3

    // Affichage des éléments
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", tableau[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

- Méthode dynamique :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int lignes = 2, colonnes = 3;
```

```
    // Allocation dynamique d'un tableau 2D
```

```
    int **tableau = malloc(lignes * sizeof(int *));
```

```
    for (int i = 0; i < lignes; i++)
```

```
        tableau[i] = malloc(colonnes * sizeof(int));
```

```
    // Remplissage du tableau
```

```
    for (int i = 0; i < lignes; i++) {
```

```
        for (int j = 0; j < colonnes; j++) {
```

```
            tableau[i][j] = i + j;
```

```
        }
```

```
    }
```

```
    // Affichage
```

```
    for (int i = 0; i < lignes; i++) {
```

```
        for (int j = 0; j < colonnes; j++) {
```

```
            printf("%d ", tableau[i][j]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    // Libération de la mémoire
```

```
    for (int i = 0; i < lignes; i++)
```

```
        free(tableau[i]);
```

```
    free(tableau);
```

```
    return 0;
```

```
}
```

Utilisation et importance :

- Pourquoi est-il nécessaire d'avoir plusieurs manières de créer des tableaux ?
- Donnez des exemples de situations où chaque méthode est utile.

Situation	Méthode adaptée	Exemple
Taille connue à l'avance	Tableau statique	Grille de jeu 3x3 (morpion)

Taille inconnue avant l'exécution	Allocation dynamique	Lecture d'un fichier dont le nombre de lignes varie
Optimisation mémoire	Dynamique (malloc)	Chargement partiel de données
Programme simple et rapide	Statique	Programme d'exemple, exercice, matrice fixe

Question 1 : Quels sont les différents types d'opérations autorisées sur les pointeurs ? Expliquez leurs objectifs à l'aide d'exemples (2 points)

◆ **1) Affectation**

On peut **affecter** une adresse à un pointeur compatible.

```
int x = 10;  
int *p = &x; // p contient l'adresse de x
```

➡ **Objectif** : lier le pointeur à une variable existante.

◆ **2) Déréférencement**

On peut accéder à la **valeur pointée** par un pointeur avec l'opérateur *****.

```
printf("%d", *p); // affiche la valeur de x, donc 10
```

➡ **Objectif** : lire ou modifier la valeur stockée à l'adresse pointée.

◆ **3) Opérations arithmétiques**

Autorisé uniquement sur les **pointeurs de tableaux**.

- **Incrémentation** / **Décrémentement** (p++, p--)
Le pointeur avance ou recule d'un élément du type pointé.

```
int tab[3] = {1, 2, 3};  
int *p = tab;  
p++; // pointe maintenant sur tab[1]  
printf("%d", *p); // affiche 2
```
- **Addition / Soustraction d'un entier**

```
p = tab + 2; // pointe sur tab[2]  
printf("%d", *p); // affiche 3
```

➡ **Objectif** : parcourir un tableau à l'aide d'un pointeur.

◆ **4) Comparaison**

Les pointeurs d'un même tableau peuvent être **comparés** (==, !=, <, >).

```
int *p1 = &tab[0];  
int *p2 = &tab[2];  
if (p1 < p2) printf("p1 pointe avant p2\n");
```

➡ **Objectif** : déterminer la position relative d'éléments en mémoire.

Quelles sont les différences entre les mots-clés struct et union en C ? Coder en C une structure de données chanson qui comporte les éléments suivants : identifiant, titre, nom de chanteur(s)/chanteuse(s), durée de la chanson, année de la sortie, nom de compositeur/compositrice, prix. Instanciez une variable de cette structure (par exemple, les détails de votre chanson préférée). Pensez à utiliser les pointeurs. (2 points)

Une structure permet de regrouper plusieurs variables (de types différents ou non) dans un même bloc mémoire, mais chaque champ a sa propre zone mémoire.

Une union permet aussi de regrouper plusieurs variables, mais tous les membres partagent la même zone mémoire.

```
#include <stdio.h>
#include <string.h> // Ajoutez cet include pour strcpy()

struct Chanson {
    char identifiant[50];
    char titre[50];
    int annee;
    int duree;
    union Artiste {
        char chanteur[50];
        char compositeur[50];
    } a;
    union Prix {
        int prix1;
        float prix2;
    } p;
};

int main() {
    struct Chanson Chanson1;
    strcpy(Chanson1.identifiant, "POP");
    strcpy(Chanson1.titre, "WAP");
    strcpy(Chanson1.a.chanteur, "Cardi B");
    Chanson1.duree = 258;
    Chanson1.annee = 2020;
    Chanson1.p.prix1 = 69;

    return 0; // Ajoutez aussi un return
}
```

Codez en C un programme qui prend des numéros (un ou plusieurs) passés par la ligne de commande. L'objectif de code est de calculer a) Le nombre de nombres b) Le numéro le plus grand c) Le numéro le plus petit d) La moyenne N'utilisez pas scanf. (2 points)

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int max = atoi(argv[1]);
    for (int i = 2; i < argc; i++) {
        if (atoi(argv[i]) > max) {
            max = atoi(argv[i]);
        }
    }

    int min = atoi(argv[1]);
    for (int i = 2; i < argc; i++) {
        if (atoi(argv[i]) < min) {
            min = atoi(argv[i]);
        }
    }

    float som = atoi(argv[1]);
    for (int i = 2; i < argc; i++) {
        som += atoi(argv[i]);
    }
    som /= (argc-1);

    printf("Nombre de nombre %d\n", (argc-1));
    printf("Nombre le plus grand %d\n", max);
    printf("Nombre le plus petit %d\n", min);
    printf("Moyenne %f\n", som);
    return 0;
}
```

Coder en C la fonction qui permet d’afficher les n premières et les n dernières lignes d’un fichier dont le nom de fichier et la valeur n sont passés par l’utilisateur en utilisant la ligne de commande. Pensez aux commandes head et tail en Linux dont l’objectif est d’afficher les n premières et les n dernières lignes d’un fichier. (2 points)

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int fd, size;
    char content;

    if (argc != 3) {
        printf("Pas le bon nombre d'arguments.\n");
        return 0;
    }

    char* nom_du_fichier = argv[1];
    int n = atoi(argv[2]);

    int nb_lignes = 1;

    int ligne = 1;

    fd = open(nom_du_fichier, O_RDONLY); // On ouvre le fichier en lecture seule
    if (fd == -1) {
        printf("Erreur : impossible d'ouvrir le fichier '%s'\n", nom_du_fichier);
        return 1; // Ou return -1 pour indiquer une erreur
    }

    while (1) {
        size = read(fd, &content, 1); // On lit le fichier octet par octet
        if (size < 1) {
            break;
        }

        if (content == '\n') {
            nb_lignes++;
        }
    }

    close(fd);
    fd = open(nom_du_fichier, O_RDONLY); // On ouvre le fichier en lecture seule

    while (1) {
        size = read(fd, &content, 1); // On lit le fichier octet par octet
        if (size < 1) {
            break;
        }
        if (content == '\n') {
            ligne++;
        }
    }

    if (ligne <= n || ligne >= (nb_lignes - n + 1)) {
        printf("%c", content); // On affiche le contenu du fichier
    }
}

close(fd);
printf("\n");

return 0;
}
```

Sans utiliser les bibliothèques standards ou externes (par exemple string.h), écrivez le code en C qui

1. Calcule le nombre de caractères dans une autre chaîne de caractères
2. Copie une chaîne de caractères dans une autre chaîne de caractères
3. Concatène deux chaînes de caractères
4. Compare deux chaînes de caractères.

Vous devez écrire des fonctions pour chacune de ces opérations sur les chaînes de caractères. (2 points)

Dans TP2, chaine.c

```
#include <stdio.h>

int compare(char char1[], char char2[]){
    int i = 0;
    while (char1[i] != '\0' && char2[i] != '\0'){
        if(char1[i] != char2[i]){
            return 0;
        }
        i++;
    }

    if (char1[i] != '\0' && char2[i] == '\0') {
        return 1;
    }
    return 0;
}

int main () {
    char chaine1[100] = "abcdef";
    char chaine2[100] = "abcdef";
    printf("comparaison : %d\n", compare(chaine1, chaine2));
    return 0;
}
```

Quelle est la principale différence entre la boucle while et la boucle do..while ?

Coder en C la fonction void afficher_1() qui permet d'afficher les nombres inférieurs ou égaux à 1000, qui sont divisibles par 5 et non divisibles par 3 en utilisant une boucle while.

Coder en C la fonction void afficher_2() qui permet d'afficher les nombres inférieurs ou égaux à 1000, qui sont divisibles par 5 et non divisibles par 3 en utilisant une boucle do..while

1 Boucle while

- **La condition est testée avant** d'entrer dans la boucle.
- Si la condition est **fausse dès le départ**, le bloc **ne s'exécute jamais**.

2 Boucle do...while

- **La condition est testée après** l'exécution du bloc.
- Le bloc est **toujours exécuté au moins une fois**, même si la condition est fausse.

```
#include <stdio.h>

int main() {
    int i = 0;
    while (i <= 1000) {
        if (i%5 == 0 & i%3 != 0) {
            printf("%d\n", i);
        }
        i++;
    }
    return 0;
}
```

```
int main() {
    int i = 0;
    do {
        if (i%5 == 0 & i%3 != 0) {
            printf("%d\n", i);
        }
        i++;
    } while (i <= 1000);
}
```

Coder en C une structure de donnée Etudiant qui comporte les éléments suivants : Nom, Prénom, Adresse, Filière (ETI, IRC ou CGP), Modules choisis (5 maximum), la note dans chaque module.

Coder en C une structure de données Module qui comporte les éléments suivants : Nom du module, responsable du module, les intervenants dans le module.

Pour les deux structures Etudiant et Module vous pouvez utiliser les types struct et/ou union (3 points)

```
#include <stdio.h>
#include <string.h>

#define MAX_MODULES 5
#define MAX_INTERVENANTS 5

// -----
// Structure Module
// -----
typedef struct {
    char nom[50];
    char responsable[50];
    char intervenants[MAX_INTERVENANTS][50];
} Module;

// -----
// Structure Etudiant
// -----
typedef struct {
    char nom[50];
    char prenom[50];
    char adresse[100];
    char filiere[10]; // ex: "ETI", "IRC", "CGP"

    Module modulesChoisis[MAX_MODULES];
    float notes[MAX_MODULES];
    int nbModules;
} Etudiant;

// -----
// Exemple d'utilisation
// -----
int main() {
    Etudiant e1;

    // Remplissage des infos de l'étudiant
    strcpy(e1.nom, "Dupont");
    strcpy(e1.prenom, "Adélie");
    strcpy(e1.adresse, "12 rue des Lilas, Lyon");
    strcpy(e1.filiere, "IRC");
    e1.nbModules = 2;

    // Module 1
    strcpy(e1.modulesChoisis[0].nom, "Programmation C");
    strcpy(e1.modulesChoisis[0].responsable, "M. Martin");
    strcpy(e1.modulesChoisis[0].intervenants[0], "Mme Durand");
    strcpy(e1.modulesChoisis[0].intervenants[1], "M. Petit");
    e1.notes[0] = 16.5;

    // Module 2
    strcpy(e1.modulesChoisis[1].nom, "Réseaux");
    strcpy(e1.modulesChoisis[1].responsable, "Mme Leroy");
    strcpy(e1.modulesChoisis[1].intervenants[0], "M. Blanc");
    e1.notes[1] = 14.0;

    // Affichage
    printf("=== Etudiant ===\n");
    printf("Nom : %s\n", e1.prenom, e1.nom);
    printf("Filière : %s\n", e1.filiere);
    printf("Adresse : %s\n", e1.adresse);
    printf("Modules choisis :\n");

    for (int i = 0; i < e1.nbModules; i++) {
        printf("  - %s (responsable : %s) → note : %.2f\n",
            e1.modulesChoisis[i].nom,
            e1.modulesChoisis[i].responsable,
            e1.notes[i]);
    }

    return 0;
}
```


Suite de Fibonacci en récursif et itératif

Réursive

```
#include <stdio.h>

// Fonction récursive pour calculer le no terme de Fibonacci
int fibonacci(int n) {
    if (n == 0) return 0;    // Cas de base : U0 = 0
    if (n == 1) return 1;    // Cas de base : U1 = 1
    return fibonacci(n - 1) + fibonacci(n - 2); // Récurrence : Un = U(n-1) + U(n-2)
}

int main() {
    int n;
    printf("Combien de termes voulez-vous générer ? ");
    scanf("%d", &n);

    printf("Voici la suite de Fibonacci (récursive) :\n");
    for (int i = 0; i < n; i++) {
        printf("U%d = %d\n", i, fibonacci(i));
    }

    return 0;
}
```

Itératif

```
#include <stdio.h>

int main() {
    int n;
    printf("Combien de termes voulez-vous générer ? ");
    scanf("%d", &n); // On demande à l'utilisateur de saisir le nombre de termes à générer
    printf("Voici la suite de Fibonacci :\n");
    int U0 = 0; // Premier terme de la suite
    int U1 = 1; // Deuxième terme de la suite
    printf("U0 = %d\n", U0);
    printf("U1 = %d\n", U1);
    int Un_2 = U0; // U(n-2)
    int Un_1 = U1; // U(n-1)
    for (int i=2; i<n; i++) {
        int Un = Un_1 + Un_2; // Calcul du terme courant U(n) = U(n-1) + U(n-2)
        printf("U%d = %d\n", i, Un); // On affiche le terme courant
        Un_2 = Un_1; // Actualisation de U(n-2)
        Un_1 = Un; // Actualisation de U(n-1)
    }
    return 0;
}
```

Une des bonnes pratiques de programmation en C est de diviser le code en plusieurs modules (module1, module2, etc...) Pour chaque module on associe un fichier .h (exemple module1.h) et un fichier .c (exemple module2.c). Quel code est implémenté dans le fichier .h ? Quel code est implémenté dans le fichier .c ? Quelle commande de compilation permet d'obtenir le fichier .o à partir du fichier.c ? Donner la commande pour avoir le module1.o à partir du module1.c Que contient le fichier module1.o ? Est-ce que le fichier module1.o est un fichier exécutable ?

◆ 1 Contenu du fichier .h (header file)

Le fichier .h contient :

- Les **déclarations** (pas les définitions) :
 - des **fonctions** (leurs prototypes)
 - des **structures, constantes, macros**
 - des **variables externes**
 - les **gardes d'inclusion** (#ifndef, #define, #endif)

Exemple – module1.h

```
#ifndef MODULE1_H
#define MODULE1_H

// Déclaration de fonction
void afficherMessage();

// Déclaration de constante
#define PI 3.14159

#endif
```

◆ 2 Contenu du fichier .c (code source)

Le fichier .c contient :

- Les **définitions** des fonctions déclarées dans le .h
- Le **code exécutable** (implémentations, algorithmes)

Exemple – module1.c

```
#include "module1.h"
#include <stdio.h>

void afficherMessage() {
    printf("Bonjour Adélie, ceci vient de module1.c !\n");
}
```

◆ 3 Compilation en fichier objet .o

Pour compiler un fichier .c sans l'exécuter, on utilise l'option **-c** de gcc :

```
gcc -c module1.c -o module1.o
```

✓ Cela crée **module1.o** sans générer d'exécutable.

◆ 4 **Contenu du fichier .o**

Le fichier .o contient :

- Le **code machine compilé**,
 - Mais **pas encore lié** avec les autres modules ni avec la bibliothèque standard.
C'est un **fichier objet intermédiaire**.
-

◆ 5 **Est-ce un fichier exécutable ?**



Non,

module1.o **n'est pas exécutable** car il n'a pas encore été **lié**.

➔ Il doit être **assemblé (lié)** avec d'autres fichiers objets via une commande comme :

```
gcc module1.o main.o -o programme
```